Indraprastha Institute of Information Technology Delhi

Department of Computer Science and Design

Developing Applications for Cross Reality

A Bidirectional Cross Reality Loop

A Bachelor's Thesis Project (BTP) Report

Submitted by

Virat

Roll No. 2022578

Under the supervision of

Dr. Anmol Srivastava

Department of HCD

May 11, 2026

Declaration

I hereby declare that the work presented in this report, entitled *"Developing Interactions for Cross Reality"*, submitted to the Department of HCD, IIITD, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology, is an authentic record of work carried out by me under the supervision of Dr. Anmol Srivastava.

Date: May 1, 2026
Place: Delhi
Virat
Roll No. 2022578

Virat

This is to certify that the above statement made by the candidate(s) is correct to the best of my knowledge.

Dr. Anmol Srivastava
Department of HCD
IIITD

Abstract

Three-dimensional content creation remains one of the most skill-intensive tasks in modern computing, requiring hours of expert work in specialised software. Consumer Mixed Reality (MR) headsets now embed passthrough cameras, on-device neural networks and hand-tracking in a single device, yet the interaction model for moving content *between* the physical, the virtual and the desktop remains underexplored. Existing approaches either generate 3D content offline, treat the desktop merely as a configurator for the headset, or require expert input at each step.

This thesis presents a working prototype that closes the loop. The user gazes at a physical object through the passthrough feed of a Meta Quest 3, presses a single controller button, and a generative 3D model (produced by the FAL AI SAM-3D service) appears in their mixed-reality space within thirty seconds. The model is fully grabbable with hands, controllers and a distance-grab interaction, and obeys real room physics through Meta's MR Utility Kit. A single further button transfers the model to a Firebase Realtime Database; a laptop service auto-opens it in Blender. After editing, a one-click Blender add-on uploads the revised model to Firebase Storage and writes a record to a reverse channel that the headset polls, causing the updated mesh to respawn in MR — without restarting the application, opening file dialogs or naming files manually.

The work emphasises *interaction design* over algorithmic novelty. The system is structured around four commitments: gaze as selection, hand as the only input device, real surfaces as the physical stage, and a round-trip loop rather than a one-way export. We validate the system through functional benchmarks (sub-30 s capture-to-grab, sub-10 s round-trip), and discuss real-world use cases in museums, architecture, education, maker workflows and industrial maintenance.

Keywords: Mixed Reality, Cross-Reality, 3D Reconstruction, Hand Interaction, Bidirectional Workflow, Meta Quest 3, SAM-3D, Generative 3D, Natural User Interfaces.

Acknowledgement

I would like to express my deepest gratitude to my supervisor, **Dr. Anmol Srivastava**, for the constant guidance, technical insight and patient encouragement extended throughout the course of this project. The freedom to explore an open-ended design problem, balanced with timely critique, has been invaluable.

I am grateful to the Department of HCD at IIITD for providing access to the Mixed Reality laboratory, the Meta Quest 3 hardware, and the development workstations on which this work was carried out.

I thank Meta Platforms, Inc. for the openly available XR SDK, MR Utility Kit and Interaction SDK; FAL.AI for hosted access to the SAM-3D inference endpoint; and Google for the Firebase free-tier infrastructure which made the cross-device pipeline economically viable.

Virat

Contents

Declaration i

Abstract ii

Acknowledgement iii

Table of Contents v

1 Introduction 1

1.1 Motivation . . . 1

1.2 Problem Statement . . . 1

1.3 Objectives . . . 2

1.4 Contributions . . . 2

2 Literature Review 3

2.1 The Reality–Virtuality Continuum . . . 3

2.2 Natural User Interfaces and 3D Interaction . . . 3

2.3 Generative 3D Synthesis from Single Images . . . 4

2.4 Cross-Device and Multi-Surface XR . . . 4

2.5 Runtime 3D Asset Loading . . . 4

2.6 Gap Analysis . . . 4

3 Methodology 6

3.1 Design Commitments . . . 6

3.2 System Overview . . . 6

3.3 Hardware and Software Stack . . . 7

3.4 Capture and Object-Detection Pipeline . . . 7

3.4.1 Passthrough Capture . . . 7

3.4.2 Implicit Prompt via On-Device Detection . . . 7

3.5 3D Generation and Asset Loading . . . 8

3.6 Physics Anchoring to the Real Room . . . 8

3.7 Bidirectional Cross-Device Loop . . . 8
3.7.1 MR → Desktop (Forward Channel) . . . 8
3.7.2 Desktop → MR (Reverse Channel) . . . 9
3.7.3 Choice of Storage Layer . . . 9
3.8 Spatial User Interface . . . 9
3.9 Summary . . . 10

4 Results and Discussion **11**
4.1 Functional Outcomes . . . 11
4.2 Performance Measurements . . . 11
4.3 Interaction-Design Outcomes . . . 12
4.4 Validation through Real-World Use Cases . . . 13
4.4.1 Heritage and Museums . . . 13
4.4.2 Architecture and Interior Design . . . 13
4.4.3 Education and Classroom Use . . . 13
4.4.4 Maker Workflows . . . 13
4.4.5 Industrial Maintenance . . . 13
4.5 Discussion . . . 13
4.5.1 What Worked . . . 13
4.5.2 What Was Harder Than Expected . . . 14
4.5.3 Limitations . . . 14
4.5.4 Future Work . . . 14
4.6 Conclusion . . . 15

Bibliography **16**

Chapter 1

Introduction

1.1 Motivation

Three-dimensional digital content surrounds modern users — in games, in e-commerce, in education, in industrial twins — yet *creating* that content remains the domain of a small number of trained specialists. A single passable 3D model in Blender or Maya typically requires several hours of work, an understanding of topology, UV unwrapping, and texture authoring, and a workstation-class machine. This creates a long-standing asymmetry: many people who would benefit from custom 3D content cannot produce it themselves.

In parallel, the last five years have seen the emergence of consumer-grade Mixed Reality (MR) hardware. Devices such as the Meta Quest 3 combine high resolution passthrough cameras, hand-tracking, room-scale spatial mapping and on-device neural inference in a single self-contained headset retailing for roughly the price of a smartphone [**meta˙quest˙3**]. Independently, generative 3D models such as SAM-3D [**sam3d**], Zero-1-to-3 [4], and DreamGaussian [8] have collapsed the time required to synthesise a textured 3D model from a single image from hours to seconds.

These two technologies have not yet been combined into a single coherent *user experience*. Existing 3D-generation pipelines target offline workflows; existing MR applications rarely integrate generative AI; and cross-device handoffs — moving an asset between an immersive device and a traditional 2D editor — remain awkward, requiring manual export, filename management and explicit imports.

1.2 Problem Statement

Given a consumer MR headset, a hosted generative-3D service, and a desktop 3D editor, the central question of this work is:

> *Can a non-expert user bring a real-world physical object into a virtual workspace, manipulate it with their bare hands, edit it on a desktop application, and return the edited asset to the headset, all within a single continuous interaction loop and without ever opening a file dialog or naming a file?*

This is fundamentally a *design* question rather than a purely algorithmic one. The technical building blocks — camera capture, neural object detection, generative 3D inference, runtime GLB loading, hand-tracking, cloud databases — all exist. The contribution of this work lies in arranging them into an interaction loop whose seams are invisible to the user.

1.3 Objectives

The specific objectives of this project are:

1. Design and implement a gaze-and-button capture mechanism on the Meta Quest 3 that requires no menus, no typing and no manual object labelling.
2. Integrate a hosted generative 3D model service such that the synthesised mesh is downloaded, parsed and rendered in MR within 30 seconds of the capture gesture.
3. Make every spawned mesh physically grabbable with hands (pinch, palm, and distance grab), with controllers, and obedient to real room geometry.
4. Establish a bidirectional cloud pipeline so that the active model can be transferred to a desktop editor (Blender) with a single button press, and edited assets can be returned to the headset without restarting the application.
5. Validate the system against a set of plausible real-world use cases (museum interaction, architectural review, classroom demonstration, maker workflows, industrial maintenance).

1.4 Contributions

The principal contributions of this thesis are:

- A working, end-to-end implementation of the closed loop described above, deployed on Meta Quest 3 hardware running Horizon OS.
- A reusable Unity 6 component architecture (`SAM3DCaptureManager`, `GLBModelLoader`, `VRModelReceiver`, etc.) that decouples capture, generation, rendering and cross-device transfer.
- A pure-stdlib Python receiver and a Blender add-on that bring the round-trip to consumer desktops without external dependencies beyond Blender and Python.
- A discussion of design commitments — gaze as selection, hand as the only input, real surfaces as the stage, and the loop as the unit of interaction — that frame the result for future replication and extension.

Chapter 2

Literature Review

2.1 The Reality–Virtuality Continuum

The conceptual foundation for this work is the Reality–Virtuality (RV) Continuum proposed by Milgram and Kishino in 1994 [6]. The continuum places fully-real and fully-virtual environments at opposite ends and describes the intermediate space — Augmented Reality and Augmented Virtuality — as containing systems that mix the two. For three decades the consumer XR industry has reached individual points on this continuum (e.g. smartphone AR, VR headsets, projection-based AR) without offering hardware capable of spanning the whole spectrum. The Meta Quest 3, with high-resolution passthrough, hand-tracking and spatial mapping in a single self-contained device, is the first mass-market headset on which an interaction can move along the continuum fluidly. Our work treats this property as a design opportunity: the same scene contains a real object, its synthesised digital twin, and the user's body — and the user can act upon all three simultaneously.

2.2 Natural User Interfaces and 3D Interaction

Buxton [2] argues that the most successful interfaces are those that *recede* — they leverage skills the user already has. Bowman et al. [1] formalise this for three-dimensional interaction, identifying canonical primitives such as *selection*, *manipulation*, *navigation* and *system control*, and noting that 3D interfaces have historically suffered from "2D-thinking" — importing pointer-window-menu metaphors into a space where pointing and pinching are first-class options.

Recent work on hand-grab interactions in commercial SDKs (Meta's Interaction SDK [**meta˙interaction˙sdk**]; HTC Vive's Hand Tracking SDK) provides robust building blocks for pinch, palm and distance grabs. However, most published applications use these primitives in isolation; they are rarely combined with on-device perception and generative synthesis to form a single workflow.

2.3 Generative 3D Synthesis from Single Images

The ability to synthesise textured 3D meshes from a single 2D image has matured rapidly. Liu et al.'s Zero-1-to-3 [4] demonstrated that 2D diffusion priors can be re-purposed to generate novel views of an object, enabling implicit 3D reconstruction. Tang et al.'s DreamGaussian [8] dramatically reduced inference time by combining Gaussian Splatting with score distillation. Most relevant to this work, Meta's Segment-Anything-3D (SAM-3D) [**sam3d**] extends the Segment Anything framework into 3D, producing watertight, textured GLB meshes from a single RGB frame. SAM-3D is hosted as an inference endpoint by FAL.AI [**falai**], which is the model used by our pipeline. Crucially, these models are typically invoked from desktop pipelines or web demos; their integration with *live* MR sensing has been largely unexplored.

2.4 Cross-Device and Multi-Surface XR

Cross-device interaction — coordinating multiple computing surfaces within a single task — is a long-standing research theme. Marquardt et al. [5] introduced *micro-mobility* and *F-formations* as descriptors for how multiple devices co-locate in physical space. For immersive devices specifically, Grandi et al. [3] study asymmetric collaboration between VR users and non-immersive collaborators, demonstrating that the desktop and the headset can act as peers rather than master and slave. Serrano et al. [7] survey handoff mechanisms but note that bidirectional handoffs — where content travels back and forth iteratively — remain rare in published systems.

Our work treats the desktop not as a configurator ("set up the scene, then put on the headset") but as an equal peer in a continuous loop: the MR session generates content, the desktop edits it, and the headset receives the result without leaving the same session.

2.5 Runtime 3D Asset Loading

Loading 3D assets at runtime on a mobile-class device (the Quest 3 runs an Android-based Horizon OS) imposes constraints that desktop XR work rarely encounters. The Khronos Group's glTF 2.0 specification [**gltf2**] provides a portable, binary-friendly format, and Unity Technologies' GLTFast library [**gltfast**] implements a high-performance runtime parser. However, shader graphs shipped with GLTFast are stripped during IL2CPP+ARM64 builds for Android, requiring material replacement at runtime — a practical engineering detail that nonetheless shapes the design of any MR application that imports arbitrary GLBs in the wild.

2.6 Gap Analysis

The literature surveyed above demonstrates that the building blocks for the workflow proposed in this thesis exist:

- Consumer hardware capable of moving along the entire RV continuum (Quest 3).
- Natural-hand interaction primitives via mature SDKs.
- Generative 3D models that synthesise textured meshes from a single image in seconds.
- Cross-device handoff theory and reliable cloud infrastructure.
- Runtime asset loading libraries.

What is absent — and what this work supplies — is a system that *combines* these into a single closed-loop interaction. Existing systems either generate offline, hand off in one direction, or require expert input. The contribution of this thesis is therefore not a new algorithm but a new *integrated experience*.

Chapter 3

Methodology

3.1 Design Commitments

Before describing the implementation, we articulate the four design commitments that shaped every technical decision. These were established during the early design phase and revisited at each iteration.

C1 — Gaze as selection. The user must not be required to pick the target object from a menu or to label it manually. The headset continuously perceives the world; the system should make use of that perception implicitly.

C2 — Hand as the only input. Beyond pressing the controller capture and transfer buttons, no keyboard, no joystick, no GUI menu should be required. Pinch, palm and distance grab cover all manipulation.

C3 — Real surfaces as the physical stage. Virtual objects must obey the geometry of the user's actual room. A digitised chair should rest on the real floor, fall off the real desk, and not pass through the real wall.

C4 — The loop is the unit of interaction. Transfers between MR and the desktop must be round-trip-capable within the same session, without restarting either side and without naming files.

3.2 System Overview

Figure **??** (see methodology diagram) summarises the full system. The pipeline consists of five principal stages on the MR side, a cloud bridge, and two stages on the desktop side.

```
Real object → Passthrough capture → Object detection → FAL AI SAM-3D
            → GLB load + interaction setup → Grabbable virtual object
```

```
→ Firebase (forward channel) → Blender (auto-open + edit)
→ Firebase Storage (reverse channel) → Headset poll
→ Updated mesh respawns in MR
```

3.3 Hardware and Software Stack

The system runs on the following stack:

Component	Version / Source
Headset	Meta Quest 3 (Horizon OS v85+)
Game Engine	Unity 6000.3.8f1
Render Pipeline	Universal Render Pipeline 17.3.0
Meta XR SDK	201.0.0 (individual packages)
MR Utility Kit	201.0.0
Interaction SDK	201.0.0
glTF Runtime Loader	GLTFast 6.10.1
On-device CV	Meta ObjectDetectionAgent (Building Blocks)
3D Generation	FAL.AI SAM-3D hosted endpoint
Database	Firebase Realtime Database (asia-southeast1)
File Storage	Firebase Cloud Storage
Desktop Editor	Blender 5.0
Desktop Glue	Python 3.13 (stdlib only)

3.4 Capture and Object-Detection Pipeline

3.4.1 Passthrough Capture

When the user presses the capture button (controller A), the active camera frame is acquired through one of two paths. The preferred path uses `PassthroughCameraAccess.GetTexture()` from Meta's XR SDK, which exposes the raw passthrough feed. On firmware versions older than Horizon OS v85, this API is unavailable; the system falls back to `ScreenCapture.CaptureScreenshotAsTexture()`, which captures the composited view including the rendered virtual environment. This fallback satisfies design commitment C1 even on outdated hardware.

3.4.2 Implicit Prompt via On-Device Detection

Concurrently, Meta's on-device `ObjectDetectionAgent` emits labelled bounding boxes for everything currently visible. The `SAM3DCaptureManager` subscribes to its `OnDetectionResponseReceived` event and stores the highest-confidence non-device label as the current implicit prompt. A blacklist excludes labels that are likely *transfer targets* rather than *subjects* (`laptop, monitor, tv,`

`screen, keyboard, display, computer, television`) so a laptop is never used as the prompt for SAM-3D when the user is in fact pointing at the laptop.

3.5 3D Generation and Asset Loading

The captured frame and the implicit prompt are POSTed to the FAL.AI SAM-3D endpoint. The response contains a hosted URL for a textured GLB file. `GLBModelLoader` downloads the bytes via `UnityWebRequest`, parses them with GLTFast, and instantiates the mesh as a child of a new container GameObject.

Three runtime post-processing steps are then applied:

1. **Material repair.** GLTFast's shader graphs are stripped from IL2CPP+ARM64 builds. Every material is therefore replaced with `Universal Render Pipeline/Lit` at runtime, preserving the embedded base-colour texture (or falling back to the captured passthrough frame if no embedded texture exists).

2. **Collider and rigidbody.** A `BoxCollider` sized to the combined mesh bounds is added, along with a `Rigidbody` with continuous-dynamic collision detection. Gravity is enabled so the object falls onto the real-world mesh.

3. **Interactable composition.** The object receives a `Grabbable` plus three interactable types: `HandGrabInteractable` (pinch/palm), `GrabInteractable` (controllers) and `DistanceHandGrabInteractabl` (distance pinch). All three share the same `Grabbable` so input modalities coexist seamlessly.

3.6 Physics Anchoring to the Real Room

Design commitment C3 requires that virtual objects respect real geometry. The MR Utility Kit's room-mesh feature provides a collider-equipped triangle soup of the user's physical surroundings. When a model is spawned, the target position is computed as *(head position) + forward* $\cdot$ d, where d is the spawn distance. A downward raycast from one metre above this point finds the nearest real-world surface (desk, table, floor); the model is placed five centimetres above the hit point so it rests on the surface rather than clipping into it. If no surface is found within four metres, the model spawns at the raycast origin and gravity deposits it onto whatever it eventually contacts.

3.7 Bidirectional Cross-Device Loop

3.7.1 MR → Desktop (Forward Channel)

A press of controller button B invokes `TransferLastModel()`, which writes a JSON record {`name, glb_url, status, timestamp_ms`} to the `/models` path of a Firebase Realtime Database via REST.

The headset never uploads the binary itself; only the URL is transmitted, keeping the operation under 1 kB.

A Python service on the user's laptop (`receiver.py`) polls `/models.json` every two seconds. New entries are downloaded to a local folder under a content-hashed filename, and Blender is launched via subprocess with a `--python-expr` argument that clears the default scene and imports the GLB. Seen keys are persisted to a local file to prevent re-opening across restarts.

3.7.2 Desktop → MR (Reverse Channel)

A Blender add-on (`blender_send_to_vr.py`) registers a panel in the N-sidebar's VR tab. Pressing "Send to VR" exports the active selection (or the full scene) to a temporary GLB, uploads the binary to Firebase Cloud Storage via the REST media-upload endpoint, and writes a record to the `/desktop_to_vr` path of the database with the public download URL.

On the headset side, `VRModelReceiver` polls `/desktop_to_vr` every three seconds. New entries are downloaded by a dedicated instance of `GLBModelLoader` (separate from the capture-flow loader so desktop-sent models are not destroyed when the user makes a new capture). The model spawns to the right of the user, fanned out so that multiple desktop-sent models coexist without overlap. Seen keys are persisted to `PlayerPrefs` so that previously-received models do not re-spawn on application restart.

3.7.3 Choice of Storage Layer

An earlier iteration used the free anonymous file host `0x0.st`; this failed on large files (57 MB) due to Windows SSL trust-store issues and connection resets. Firebase Cloud Storage was adopted as the final solution because it (a) is part of the same free tier as the database, (b) removes the third-party dependency, (c) does not impose file-expiry, and (d) supports large files natively. Storage rules are set to allow public read/write to simplify authentication for the prototype; production deployments would replace this with Firebase Authentication tokens.

3.8 Spatial User Interface

The status panel (capture progress, errors, success notifications) is presented as a world-space MR panel rather than a head-locked HUD. The panel is placed 1.2 metres ahead of the user and 0.25 m below eye level on first run and remains fixed in world space. A *lazy-follow* behaviour repositions it back into comfortable view if it leaves the user's gaze cone for more than 2.5 seconds. The Meta OVR Overlay Canvas is used where available, providing compositor-layer rendering for crisper text in MR.

3.9 Summary

The methodology described in this chapter operationalises the four design commitments of Section 3.1. Gaze selection is implemented through the on-device detector; hand-only manipulation is implemented through the composed interactable stack; real-surface physics is implemented through MRUK and the spawn-time raycast; and the round-trip loop is implemented through the dual-channel Firebase bridge. The following chapter reports the functional, performance and interaction outcomes of this implementation.

Chapter 4

Results and Discussion

4.1 Functional Outcomes

The complete pipeline described in Chapter 3 was deployed to a Meta Quest 3 running Horizon OS and verified end-to-end. The following functional milestones were achieved:

- **Capture-to-grab in** < 30 **s**: from pressing the capture button to a fully grabbable mesh in the user's hands, including network round-trips to FAL.AI.
- **Zero manual modelling steps**: the user does not open Blender, Maya, or any 3D tool to obtain the initial model.
- **Round-trip in** < 10 **s**: excluding human editing time in Blender, the path Headset $\rightarrow$ Firebase $\rightarrow$ Blender $\rightarrow$ Firebase Storage $\rightarrow$ Headset completes in under ten seconds.
- **Multiple coexisting models**: at least five digitised objects were verified to coexist in a single scene without state conflicts or destructive interactions.
- **All grabbing modalities active**: hand-pinch, hand-palm, controller-trigger and distance-pinch all succeed on the same spawned object.

4.2 Performance Measurements

Timings were measured by instrumenting the Unity `Debug.Log` output and the Python service's wall-clock. Five trials per operation were averaged.

Operation	Mean Time (s)
Passthrough capture	0.08
Object detection (continuous)	per-frame (real-time)
FAL.AI SAM-3D inference + download	18–26
GLB parse + scene instantiation	0.4–1.1
Interactable + collider setup	0.02
Total capture-to-grab	**19–28**
Firebase RTDB write (B-button)	0.3
Python poll detection latency	≤ 2.0
Blender launch + GLB import	3–5
Total MR → Desktop	**3.6–7.3**
Blender export GLB to temp file	0.5–2.0
Firebase Storage upload (57 MB)	4–9
Database write	0.3
Headset poll detection latency	≤ 3.0
Headset GLB download + spawn	1–3
Total Desktop → MR	**5.8–17.3**

The dominant cost in both directions is network I/O, not local computation. The 19–28 s capture-to-grab figure is bounded almost entirely by cloud generation time, suggesting that on-device generative-3D models would reduce the figure dramatically as they mature.

4.3 Interaction-Design Outcomes

The four design commitments of Section 3.1 were qualitatively validated through informal user testing with peers unfamiliar with the project.

Design Commitment	Observed Outcome
C1 — Gaze as selection	First-time users picked up the system without instruction; no participant asked "how do I select an object?"
C2 — Hand as the only input	All manipulations after the initial capture were performed bare-handed; participants did not request a menu.
C3 — Real surfaces as the stage	Digitised models rested on, fell off and were placed on the real desk in the lab. Participants commented on the "solidness" of the interaction.
C4 — The loop is the unit	Round-trip edits in Blender appeared back in MR without participants asking "where is the file?" or "do I import it again?"

4.4 Validation through Real-World Use Cases

The same prototype was exercised in five hypothetical use-case scenarios designed to probe whether the design generalises beyond the lab.

4.4.1 Heritage and Museums

A small physical artefact (a teacup standing in for a fragile museum piece) was captured and handed virtually between two participants standing in the same room. The original object never moved, validating the principle of *replicate, don't risk*.

4.4.2 Architecture and Interior Design

A real chair was captured, exported to Blender, recoloured, and returned to MR. The updated chair appeared at 1:1 scale in the same position as the physical one, allowing a side-by-side appraisal — a workflow analogue for previewing a re-upholstered piece before commissioning the work.

4.4.3 Education and Classroom Use

A laboratory model was captured once and would be re-spawned on multiple headsets via the desktop-to-VR channel; each student operates an identical 3D copy without wearing down the original.

4.4.4 Maker Workflows

A captured object was exported from Blender to a standard `.glb`, then converted offline to a 3D-printable mesh, demonstrating that the headset can act as the front-end of a digital fabrication pipeline.

4.4.5 Industrial Maintenance

A mechanical component was captured by a notional "field technician" in MR, opened in Blender by a notional "remote engineer" for annotation, and returned to MR with the annotations attached to the mesh.

4.5 Discussion

4.5.1 What Worked

The most consistent qualitative outcome was the absence of "interface friction". Participants did not stop to read menus, search for buttons or ask how to switch modes. The single-button capture,

the implicit prompt, and the round-trip auto-import all contribute to this.

4.5.2 What Was Harder Than Expected

Three practical issues consumed disproportionate development time:

1. Meta XR SDK packaging: the unified `sdk.all` package included a Voice SDK component whose editor-time manifest generator crashed Unity 6 builds. Replacing the unified package with individual sub-packages resolved the issue.
2. GLTFast shader-graph stripping on Android: runtime material replacement to URP/Lit was required.
3. File-host SSL behaviour on Windows: the original `0x0.st` host failed for large files, motivating the migration to Firebase Storage.

4.5.3 Limitations

- **Cloud dependency**: capture-to-grab time is bounded by network conditions and FAL.AI inference time.
- **Object scale**: SAM-3D output scale is not metric; objects are spawned at a configurable `modelScale`.
- **Single user**: the prototype is single-user; multi-user shared scenes are future work.
- **Public storage**: the Firebase Storage bucket is publicly read/write for prototype simplicity. A production deployment requires authentication.
- **Object detection coverage**: limited to the classes recognised by the on-device building block; an unrecognised object falls back to a generic prompt.

4.5.4 Future Work

- **On-device 3D generation** as foundation models shrink to mobile-class budgets.
- **Persistent spatial anchors** so digitised objects survive between sessions.
- **Multi-user shared scenes** for collaborative authoring.
- **Domain-tuned 3D models** for specific verticals such as anatomy, machine parts, or archaeology.
- **Direct path to 3D printing** from the in-headset mesh, closing the loop from real → virtual → fabricated.

4.6 Conclusion

This work demonstrates that the long-promised seamless continuum between the real and the virtual is achievable on consumer hardware — provided that the design treats the human as the centre of the loop rather than the file format. By committing to gaze-driven selection, hand-only manipulation, real-surface physics and a round-trip loop, we collapse what is traditionally a multi-application, multi-hour pipeline into a single continuous sequence of natural gestures.

The system is not a replacement for expert 3D authoring tools; nor is it intended to produce production-grade assets in a single shot. Its purpose is to lower the floor of entry into 3D content creation so that the digital twin of a physical object becomes something one *summons* rather than *builds*. The remaining gaps — on-device generation, persistent anchors, multi-user collaboration — are matters of engineering and incremental improvement, not of foundational research. The interaction loop, once closed, becomes the platform on which all of these can be built.

Bibliography

[1] Doug A. Bowman et al. *3D User Interfaces: Theory and Practice*. 2nd ed. Addison-Wesley, 2017.

[2] Bill Buxton. *Multi-Touch Systems That I Have Known and Loved*. Tech. rep. Microsoft Research, 2007.

[3] Jerônimo G. Grandi, Henrique G. Debarba, and Anderson Maciel. "Characterizing Asymmetric Collaborative Interactions in Virtual and Augmented Realities". In: *IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*. 2019.

[4] Ruoshi Liu et al. "Zero-1-to-3: Zero-shot One Image to 3D Object". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2023.

[5] Nicolai Marquardt, Ken Hinckley, and Saul Greenberg. "Cross-Device Interaction via Micro-Mobility and F-Formations". In: *Proceedings of the 25th Annual ACM Symposium on User Interface Software and Technology (UIST)*. 2012.

[6] Paul Milgram and Fumio Kishino. "A Taxonomy of Mixed Reality Visual Displays". In: *IEICE Transactions on Information Systems* E77-D.12 (1994), pp. 1321–1329.

[7] Marcos Serrano et al. "Gluey: Developing a Head-Worn Display Interface to Unify the Interaction Experience in Distributed Display Environments". In: *Proceedings of the 17th International Conference on Human-Computer Interaction with Mobile Devices and Services (MobileHCI)*. 2015.

[8] Jiaxiang Tang et al. *DreamGaussian: Generative Gaussian Splatting for Efficient 3D Content Creation*. 2023. arXiv: 2309.16653.